

CSE-1221
Computer Programming-II
FINAL

❖ Operators which can't be overloaded

(.) (.*) (::) (?:)

- ❖ The precedence of an operator can't be changed by overloading. However, parentheses can be used to force the order of evaluation of overloaded operators in an expression.
- ❖ The associativity of an operator can't be changed by overloading.
- ❖ When overloading (), [], -> or any of the assignment operators, the operator overloading function must be declared as a class member. For the other operators, the operator overloading function can be non-member functions.

❖ Overloading unary not(!) using member function

```
class Test {  
public:  
    bool operator!() const;  
    .....  
};
```

❖ Overloading Unary Not(!) using friend function

```
class test {  
public:  
    friend bool operator! (const Test&);  
    .....  
};
```

❖ Overloading Binary += Operator

Using member function

```
class test {  
public:  
    const Test &operator += (const Test&);  
    .....  
};
```

Using friend function

```
class test {  
public:  
    friend const Test &operator+= (Test&, const Test&);  
    .....  
};
```

❖ Overloading Binary Minus(-) operator using friend function

```
#include<iostream>  
using namespace std;  
class Test {  
    int a, b;  
public:  
    Test () { a = b = 0; }  
    Test (int x, int y) { a=x; b=y; }  
    void show() { cout << "a= " << a << " b= " << b << endl; }  
    friend Test operator-(Test &t1, Test &t2);  
};  
Test operator-(Test &t1, Test &t2) {  
    Test temp;  
    temp.a = t1.a - t2.a;  
    temp.b = t1.b - t2.b;  
    return temp;  
}  
int main() {  
    Test A(5, 10), B(10, 3), C;  
    C = A - B;  
    C.show();  
    return 0;  
}
```

Output: a= -5 b= 7

❖ Overload Stream.extraction (>>) and stream-insetion (<<) operators

```
#include<iostream>  
using namespace std;  
class OOP {  
    int p, q;  
public:  
    OOP () {p = q = 0;}  
};
```

```

    OOP(int i, int j) {p=i; q=j;}
    friend istream &operator >> (istream &input, OOP &obj);
    friend ostream &operator << (ostream &output, const OOP &obj);
};
istream &operator >> (istream &input, OOP &obj) {
    input >> obj.p >> obj.q;
    return input;
}
ostream &operator << (ostream &output, const OOP &obj) {
    output << "p= " << obj.p << "q= " << obj.q << endl;
    return output;
}
int main () {
    OOP test;
    cin >> test;
    cout << test;
    return 0;
}

```

- ❖ Suppose that there are 3 objects t1, t2, t3 of class string. Write a complete program that gives the output like the following and use operator overloading function:

```

t1 = New
t2 = Dhaka
t3 = New Chittagong
New Chittagong is greater than New Dhaka

```

Code:

```

#include<iostream>
#include<string>
using namespace std;
bool operator > (string s1, string s2) {
    if(s1.size() > s2.size()) return true;
    return false;
}
int main () {
    string t1 = "New", t2 = "Dhaka", t3 = "New Chittagong";
    cout << "t1= " << t1 << endl;
    cout << "t2= " << t2 << endl;
    cout << "t3= " << t3 << endl;
    if(t3 > t1 + " " + t2) {
        cout << t3 << " is greater than " << t1 + " " + t2 << endl;
    }
}

```

```
    return 0;
}
```

❖ Overloading increment operator using member function

```
#include<iostream>
using namespace std;
class data{
    int a, b;
public:
    data() {a = b = 0;}
    data(int x, int y) {a = x; b = y;}
    data &operator++() {++a; ++b; return *this;} //pre-increment
    void operator++(int) {a++; b++;} //post-increment
    void show() {cout << a << " " << b << endl;}
};
int main () {
    data d1(5,6), d2(-1,9);
    ++d1;
    d2++;
    d1.show();
    d2.show();
    return 0;
}
```

output:

```
6   7
0   10
```

❖ Overload decrement operator using friend function

```
#include<iostream>
using namespace std;
class data{
    int a, b;
public:
    data() {a = b = 0;}
    data(int x, int y) {a = x; b = y;}
    friend data &operator--(data &obj) { //pre-increment
        --obj.a;
        --obj.b;
        return obj;
    }
}
```

```

friend data operator--(data &obj, int) { //post-increment
    obj.a--;
    obj.b--;
    return obj;
}
void show() {cout << a << " " << b << endl;}
};
int main () {
    data d1(5,6), d2(-1,9);
    --d1;
    d2--;
    d1.show();
    d2.show();
    return 0;
}

```

output:

4 5
-2 8

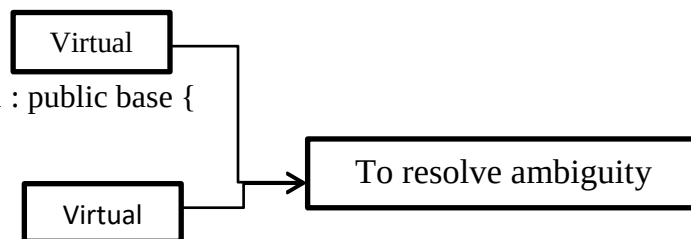
Inheritance

❖ Ambiguity in Multiple Inheritance | Uses of Virtual Base class

```

class base {
public:
    int a;
};
class derived1 : public base {
public:
    int b;
};
class derived2 : public base {
public:
    int c;
};
class derived3 : public derived1, public derived2 {
public:
    void sum() {
        cout << a + b + c << endl; //ambiguous
    }
};
int main() {

```



```

    derived3 ob;
    ob.a = 1;
    ob.b = 2;
    ob.c = 3;
    ob.sum();
    return 0;
}

```

❖ class A : public B{....}

All the public members of B are inherited to A and can be accessed directly from A.
 All the protected members of B are inherited to A can be accessed directly from A.
 All the private members of B are inherited but hidden in A and thus can not be accessed directly from A.

❖ class A : private B{....}

All the public and protected members of B are inherited to A and remain private, so they can be accessed from A directly.
 All the private members of B are also inherited to A but remain hidden in A. So, they can't be accessed directly from A.

❖ Showing the uses of Virtual Function

```

#include<iostream>
using namespace std;
class parent {
public:
    virtual void show() {cout << "I am parent" << endl;}
};
class daughter : public parent {
public:
    void show() {cout << "I am daughter" << endl;}
};
class son : public parent {
public:
    void show() {cout << "I am son" << endl;}
};
int main () {
    parent pobj;
    daughter dobj;
    son sobj;
    parent *parPtr;
}

```

```

    parPtr = &pobj;
    parPtr -> show();
    parPtr = &dobj;
    parPtr -> show();
    parPtr = &sobj;
    parPtr -> show();
    return 0;
}

```

output:

```

I am parent
I am daughter
I am son

```

Exception Handling

- ❖ Program statements that we want to monitor for exceptions are conained in a **try** block. If ans exception occus within the **try** block, it is thrown using throw. The exception is caught, using **catch**, and processed.

```

try {
    //try block
}
catch(type1 arg) {
    //catch block
}
catch(type2 arg) {
    //catch block
}
.
.
.
catch(typeN arg) {
    //catch block
}

```

- ❖ What will happen if exception is thrown for which there is no corresponding catch statement?
 - ⇒ In such cases, an abnormal program termination might occur. If the compiler compiles with standard c++, throwing an unhandled exception causes the standard library function terminate() to be invoked. By default, teminate() calls abort() to

stop the corresponding program, but anyone can specify his/her own termination handle.

- ❖ In modern C++, when an allocation request cannot be honored, now throws a bad-alloc exception.
- ❖ Rethrowing Mechanism

```
#include<iostream>
using namespace std;
double divide(int numerator, int denominator)
{
    try{
        if(denominator == 0) {
            throw -1;
        }
        return (double)numerator / denominator;
    }
    catch(int) {
        cout << "Catching exception inside divide()" << endl;
        throw;
    }
}

int main() {
    int x, y;
    double q;
    cin >> x >> y;
    try{
        q = divide(x,y);
        cout << q << endl;
    }
    catch(int) {
        cout << "Catching exception inside main()" << endl;
    }
    return 0;
}
```

I/O

- ❖ Stream

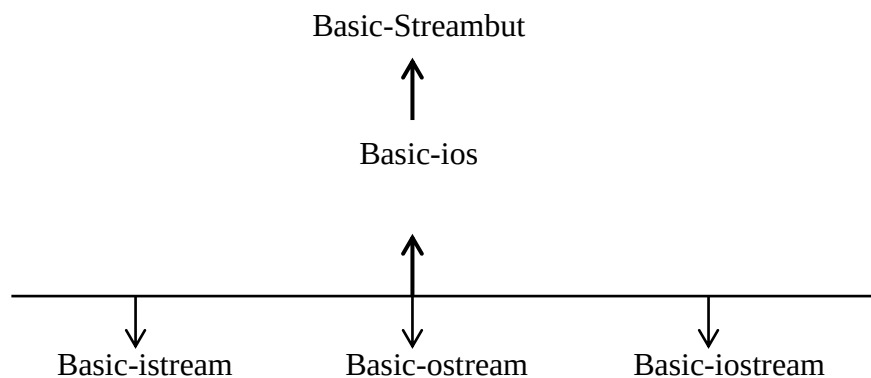
A stream is a logical device that either produces or consumes information. A stream is linked to a physical device by the C++ I/O system. All streams behave in the same manner, even if the actual physical device they are linked to differ. Because all programmer with a consistent interface, even through it operator in device with differing capabilities.

For example, the same function that we use to write to the screen can be used to write to a disk file or to the printer.

When a C program begins execution, 3 predefined streams are automatically opened : stdin, stdout, stderr. A similar thing happens when C++ program starts running. When a C++ program begins, these 4 streams are automatically opened:

Stream	Meaning	Default Device
cin	Standard input	keyboard
cout	Standard output	screen
cerr	Standard error	screen
clog	buffered version of cerr	screen

Standard C++ also opens wide (16-bit) character version of these streams called wcin, wcout, wcerr, wclog to support languages such as Bengali, Chinese etc.



❖ Format Flags

1. adjustfield
2. basefield
3. boolalpha
4. dec
5. fixed
6. floatfield
7. hex
8. internal
9. left
10. oct
11. right

12. scientific
13. showbase
14. showpoint
15. showpos
16. skipws
17. unitbuf
18. uppercase

❖ A table of square roots & squares

```
#include<iostream>
#include<cmath>
using namespace std;
int main() {
    double x;
    cout.precision(4);
    cout << "x \t\t sqrt(x) \t\t x^2" << endl;
    for(x = 1.0; x<=20.0; x++) {
        cout.width(10);
        cout << x << "\t\t";
        cout.width(10);
        cout << sqrt(x) << "\t\t";
        cout.width(10);
        cout << x * x << endl;
    }
    return 0;
}
```

❖ Importance of I/O manipulators

1. A manipulator can consolidate a sequence of several separate I/O operations.
2. A custom manipulator could be important when we need to perform I/O operations on a non-standard device.

❖ Write a program to create a manipulator called display() that sets the field width to 8, the precision to 5 and fill character to #.

```
ostream &display(ostream &output) {
    output.width(8);
    output.precision(5);
    output.fill('#');
    return output;
}
```

```
int main() {
    cout << display << 123.345 << endl;
    return 0;
}
```

output:

###123.35

❖ Write a manipulator which provides:

- 1) 10 columns width
- 2) Right justified
- 3) Two digits precision
- 4) Fill unused places with *

```
ostream &display(ostream &output) {
    output.width(10);
    output.precision(2);
    output.fill('*');
    output.self(ios::fixed | ios::right);
    return output;
}

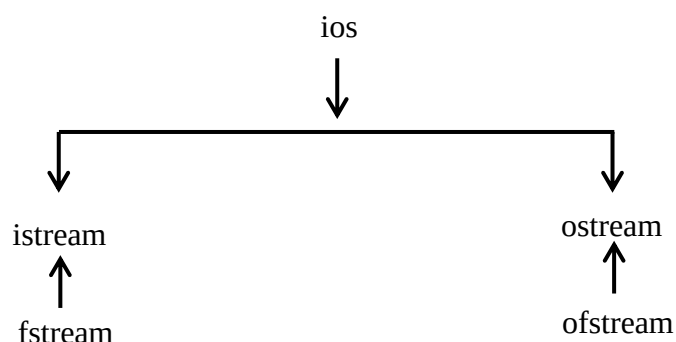
int main() {
    cout << display << 123.4567 << endl;
    return 0;
}
```

output:

*****123.46

FILE I/O

```
#include<fstream>
```



❖ File opening mode

ios::app
ios::ate
ios::binary
ios::in
ios::out
ios::trunc

❖ A program that display the contents of any file on console screen using get() function

```
#include<iostream>
#include<fstream>
using namespace std;
int main() {
    ifstream fin;
    char ch;
    fin.open("cse.txt");
    if(!fin) {
        cout << "File open error\n";
        return 1;
    }
    while(true) {
        fin.get(ch);
        if(fin.eof()) break;
        cout << ch;
    }
    return 0;
}
```